

LLM DEPLOYMENT USING KUBERNETES + OLLAMA+ MONITORING

OVERVIEW

- This project focuses on deploying an open-source LLM using Kubernetes and building an end-to-end chatbot system. A FastAPI backend processes user queries and interacts with the Mistral model via Ollama. The system is containerized using Docker and deployed on Kubernetes for scalability and reliability.
- Monitoring is implemented using Prometheus and Grafana to track system performance such as CPU and memory usage. The project also includes a comparison between open-source and commercial LLMs, demonstrating practical trade-offs in real-world deployments.

ROHIT JADHAV

NIT SILCHAR

DEVOPS / LLM ASSIGNMENT

ARCHITECTURE

User sends prompt via UI

The request is forwarded to the FastAPI backend service exposed via Kubernetes

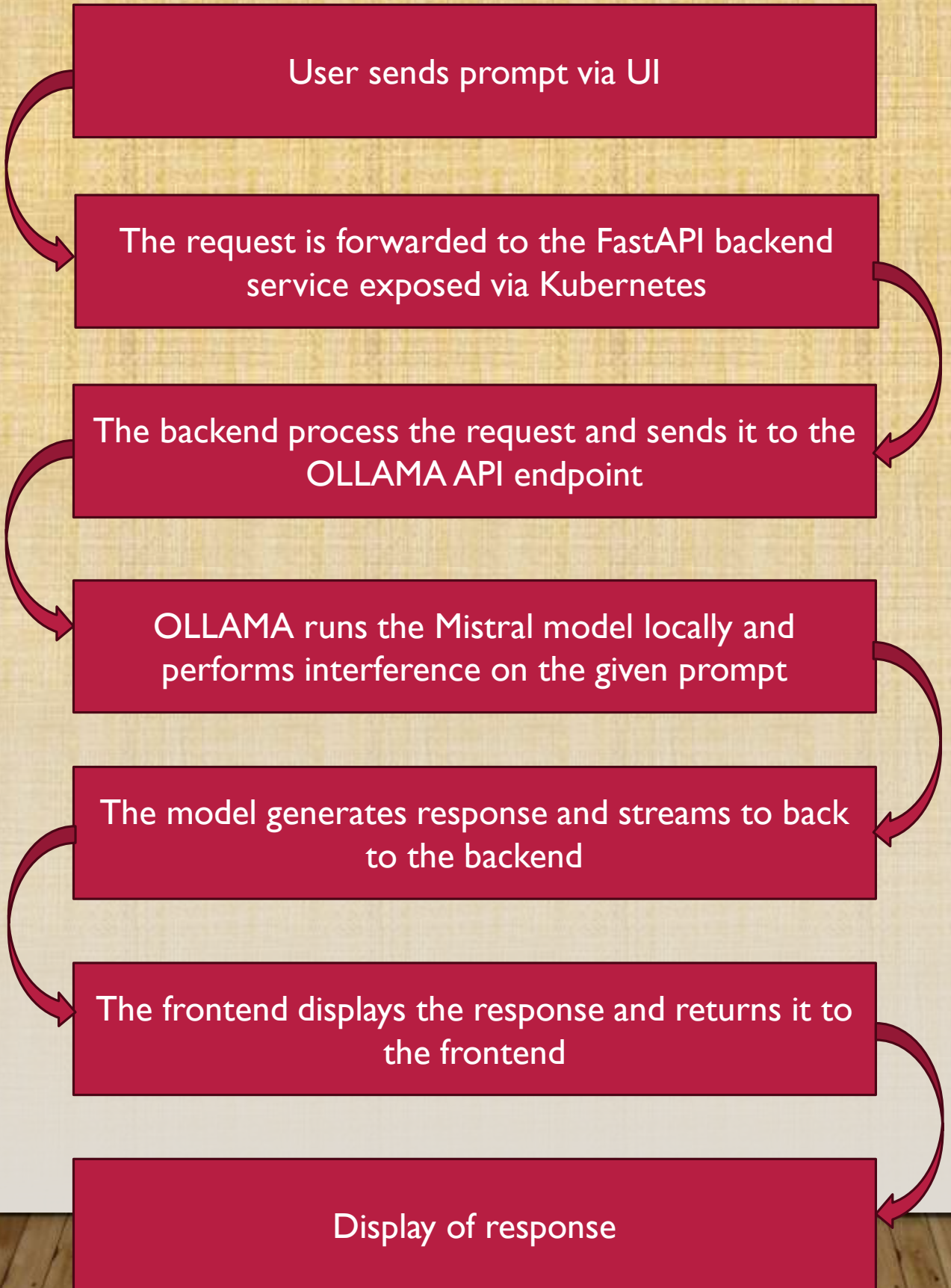
The backend process the request and sends it to the OLLAMA API endpoint

OLLAMA runs the Mistral model locally and performs inference on the given prompt

The model generates response and streams to back to the backend

The frontend displays the response and returns it to the frontend

Display of response



INTERNAL WORKING



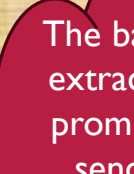
Request on /chat by
user
(Using my IP
address
192.168.219.7)



Request reaches
Kubernetes
Service
→ running Pod



Inside pod → The FastAPI application running via Uvicorn receives the request at the /chat

A red cloud shape with a black outline, containing the text "The backend extracts the prompt and sends an HTTP request to OLLAMA".

The backend extracts the prompt and sends an HTTP request to OLLAMA



Python
code
execute

```
response =
requests.post(OLLAMA_URL,
              json={
                  "model": "mistral",
                  "prompt": prompt
              },
              stream=True
            )
```



Mistral giving response



Response

[illegible]

The screenshot shows a web browser window with a purple header. The title bar says "Chat box". The header contains the text "AI Chat System" and a "Feedback" button. Below the header, there's a section titled "Introduction to AI Chat System" with a paragraph of text. A blue button labeled "Generate text or image or video" is visible. The main content area shows a chat log with a user message and a system response. At the bottom, there's a text input field and a "Send" button.

HOW IT STARTED???

Docker file
Tell set of steps
to do for Image
creation

The image shows a Dockerfile on the left and its build output on the right. The Dockerfile contains instructions like FROM, COPY, RUN, and EXPOSE. The build output shows the steps being executed, including installing dependencies and building the application.

Docker: Manual
Kubernetes:Auto

Containerized && Make Image

Kubernetes
deployment
Reads backend-
deployment.yaml
and create pods

The image shows a Kubernetes deployment.yaml file on the left and its output on the right. The deployment.yaml file contains metadata, spec, and strategy. The output shows the deployment being created and the pods being scheduled.

Inside Pod : Docker
Image Runs
fastAPI app, python
code Uvicorn serve

Service

User

IMPORTANT: POD-IP changes but
Service(LLM-SERVICE) IP is constant
That's where I got problem
Kubernetes makes and destroys PODS

MONITORING WITH GRAFANA

Prometheus

Collects real-time metrics from Kubernetes components such as CPU usage, memory consumption and network activity

Grafana

Grafana visualizes these metrics through interactive dashboards

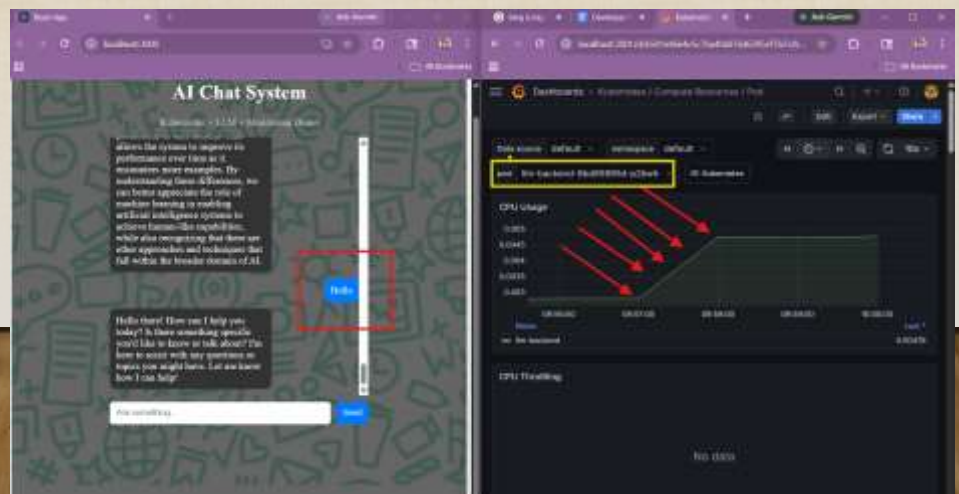
Install Prometheus
Use one version
I used kube-prometheus-stack

```
Did you mean this?  
port-forward  
PS C:\Users\johndoe> kubectl port-forward svc/prometheus-0  
ng-grafana 3001-3001  
Forwarding from 127.0.0.1:3001 -> 3001  
Forwarding from [::1]:3001 -> 3001  
Handling connection for 3001  
Handling connection for 3001  
Handling connection for 3001  
Handling connection for 3001
```

Open dashboard
and login

When user sends a prompt, the request is processed when the whole process completes, the system resources are utilized and reflected in Grafana

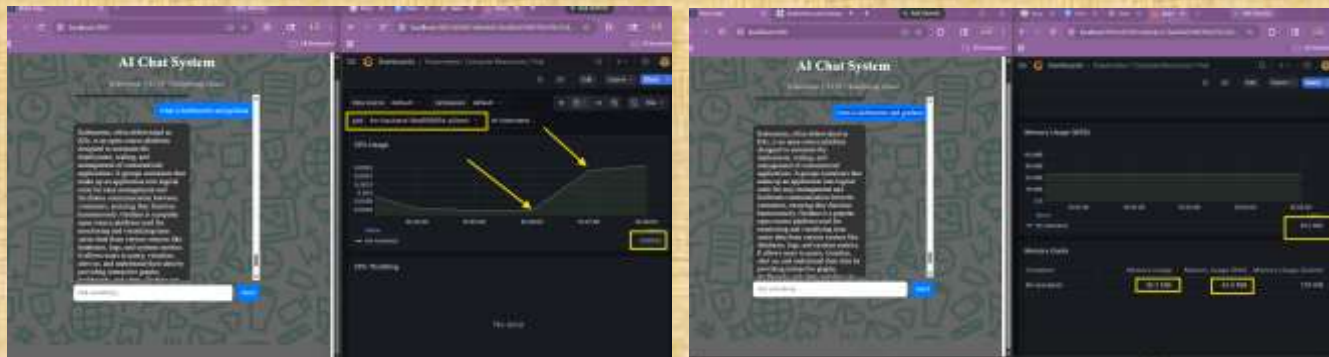
Example for
“Hello”



COMPARISON CHATGPT VS LOCALLY DEPLOYED OPEN-SOURCE LLM (MISTRAL)

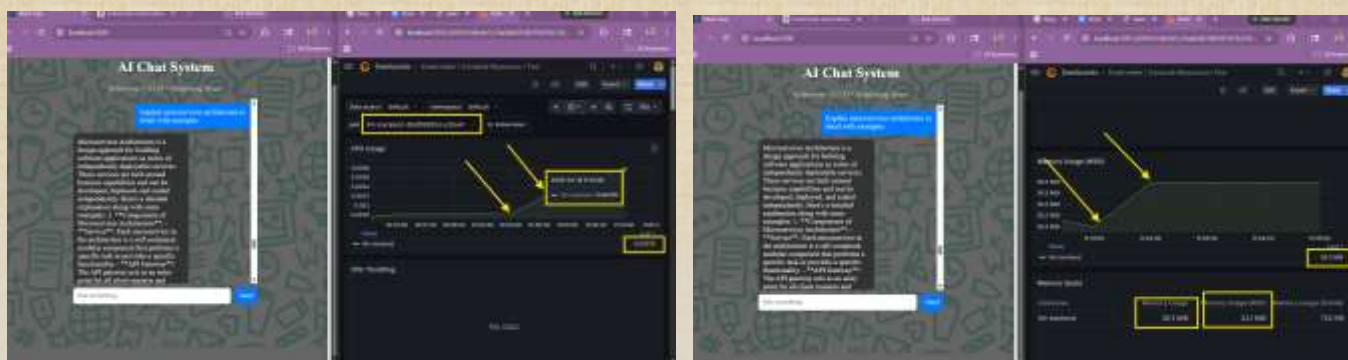
- The comparison highlights local and cloud-based LLM deployments in real world scenarios

Easy question



Model	CPU Usage	Memory Usage	speed	Response Quality	Privacy	Other Factors
Ollama Mistral	0.0033 cores	36.2MB	01 Min 01 sec 46 ms	Good	on top 🔥	Offline, CPU dependent
ChatGPT	NA	NA	12 sec 17 ms	High level	Not good	Online, optimized

Hard Question



Model	CPU Usage	Memory Usage	speed	Response Quality	Privacy	Other Factors
Ollama Mistral	0.00378 cores	36.3MB	02 min 37 sec 42ms	Better for knowledge	Again on top	Offline CPU dependent
ChatGPT	NA	Na	20 sec	in Depth	Not good	Online, No CPU needed

COST CALCULATION

Model	Cost per Query	10 query Cost	Monthly Subscription
Ollama Mistral	0	0	0
ChatGPT	~\$0.01	~\$0.10	~\$10

CONCLUSION

Open source models like Mistral are not trying to replace GPT-4. They're for use cases where privacy matters, cost matters, or you need to run offline. For general quality — commercial models still win. But for the right use case — open source is more than good enough.